

PixelForge

Advanced Digital Image Processing Platform

Nama Aplikasi	PixelForge
Subtitle	Advanced Digital Image Processing Platform
Mata Kuliah	Pengolahan Citra Digital
Dosen Pengampu	Rizki Elisa Nalawati, S.T., M.T.
Stack	React (Frontend) + Python Flask (Backend)
Versi Dokumen	1.0.0

React
18

Python
3.11

Flask

OpenCV

TensorFlow
/ Keras

Tailwind
CSS

Dokumen ini merupakan panduan lengkap pengembangan aplikasi PixelForge secara fullstack, mencakup spesifikasi fitur, arsitektur sistem, struktur proyek, panduan setup, serta referensi API endpoint untuk memudahkan proses implementasi.

1. Tentang PixelForge

PixelForge adalah aplikasi pengolahan citra digital berbasis web yang dibangun dengan arsitektur fullstack modern. Frontend menggunakan **React** untuk antarmuka yang interaktif dan responsif, sementara backend menggunakan **Python Flask** yang mengintegrasikan OpenCV, Pillow, NumPy, dan TensorFlow untuk semua operasi pengolahan citra.

■ Nama 'PixelForge' mencerminkan proses 'menempa' (forge) piksel — mengolah, membentuk, dan mentransformasi gambar digital menjadi karya yang diinginkan, layaknya seorang pandai besi yang menempa logam.

Filosofi Nama Aplikasi

Pixel	Unit terkecil gambar digital — elemen fundamental yang menjadi objek manipulasi
Forge	Menempa / membentuk dengan presisi dan kekuatan — representasi kemampuan transformasi citra

Tech Stack Ringkas

Layer	Teknologi	Peran
Frontend	React 18 + Vite + Tailwind CSS	UI interaktif, routing, state management
Backend	Python 3.11 + Flask + Flask-CORS	REST API, logika pemrosesan citra
Image Processing	OpenCV, Pillow, NumPy, SciPy	Semua operasi citra (filter, transform, segmentasi)
ML / CNN	TensorFlow 2.x + Keras	Pengenalan objek (nilai tambah)
Visualization	Matplotlib, Recharts	Histogram, chart analisis
Komunikasi	REST API + JSON + Base64	Transfer gambar antara client dan server

2. Spesifikasi Fitur Sistem

PixelForge mengimplementasikan 10 modul fitur utama sesuai spesifikasi mata kuliah, ditambah satu fitur nilai tambah berbasis Machine Learning.

- 01** **Image Management**
Load gambar (JPG, PNG, BMP), simpan dengan custom filename & format, reset ke gambar awal. Panel preview before–after real-time.
Core
- 02** **Image Enhancement**
Brightness & Contrast dengan slider interaktif, Histogram Equalization otomatis, Sharpening filter, dan Smoothing (blur).
Core
- 03** **Geometric Transformation**
Rotate 0°–360°, Flip horizontal/vertikal, Crop dengan drag area, Resize (scaling), Translation. Implementasi matriks affine dengan interpolasi nearest/bilinear.
Core
- 04** **Image Restoration**
Gaussian Blur, Median Filter, Noise removal (salt & pepper). Teknis: spatial filtering dan kernel convolution.
Core
- 05** **Binary & Edge Processing**
Thresholding (binary image), Edge Detection (Canny, Sobel, Prewitt, Robert, Laplacian, LoG), Morphology (Erosion, Dilation). Kernel structuring element.
Core
- 06** **Color Processing**
Konversi RGB → Grayscale, Channel splitting (R, G, B), Color adjustment (hue/saturation). Manipulasi color space dan channel array.
Core
- 07** **Image Segmentation**
Threshold-based, Edge-based, dan Region-based segmentation. Teknis: clustering sederhana, masking, region extraction.
Core
- 08** **Image Compression**
Save dengan kualitas berbeda (low–high), simulasi kompresi JPEG. Metode: Huffman, Aritmetik, LZW, RLE, Kuantisasi.
Core

09

Histogram Analysis

Tampilan histogram grayscale, perbandingan before–after. Distribusi intensitas pixel dengan visualisasi matplotlib.

Core

10

User Interface

Menu toolbar (File, Edit, Filter, Transform), panel preview before vs after, slider untuk parameter, tombol aksi cepat.

Core

★

CNN Object Recognition

Pengenalan objek dengan Convolutional Neural Network (nilai tambah). Pilih satu kategori objek (manusia/hewan/objek lain). Model dilatih dengan TensorFlow/Keras.

Bonus

3. Arsitektur Sistem

PixelForge menggunakan arsitektur **Client-Server** dengan komunikasi melalui REST API. Gambar dikirim dalam format **Base64** melalui JSON payload. Setiap operasi bersifat stateless — server tidak menyimpan state gambar antar request, semua state dikelola di frontend.

Layer	Komponen	Tanggung Jawab
Presentation	React Components + Tailwind	Render UI, interaksi pengguna
State	Zustand Store	Global state: gambar aktif, history, UI state
Service	imageService.js + api.js	HTTP calls ke backend, error handling
API Gateway	Flask Routes (Python)	Routing request, validasi input
Business Logic	services/*.py (OpenCV)	Semua operasi pemrosesan citra
ML Layer	TensorFlow CNN Model	Inference pengenalan objek
Utility	utils/*.py	Base64, validators, response formatter

Alur Komunikasi Data

1. User upload gambar → React mengonversi ke Base64
2. Frontend mengirim POST request ke Flask endpoint dengan payload JSON {image: base64, params: {...}}
3. Flask mendekode Base64 → numpy array → proses dengan OpenCV/PIL
4. Hasil dikonversi kembali ke Base64 → dikembalikan sebagai JSON response
5. React menerima response → update state → render gambar hasil di panel preview

✓ Semua gambar dikomunikasikan sebagai string Base64 dalam JSON. Ini memastikan tidak ada masalah CORS dengan file upload, dan memudahkan integrasi tanpa perlu file server terpisah.

4. Struktur Proyek

4.1 Frontend — React (Vite)

```

pixelforge-frontend/
├── public/
│   └── favicon.ico
├── src/
│   ├── pages/
│   │   ├── EditorPage.jsx      ← Halaman utama editor (PALING PENTING)
│   │   ├── HomePage.jsx       ← Landing page
│   │   ├── HistoryPage.jsx    ← Riwayat edit
│   │   └── NotFound.jsx
│   ├── components/
│   │   ├── Toolbar/
│   │   │   ├── ToolbarMenu.jsx ← Menu File / Edit / Filter / Transform
│   │   │   └── ToolbarActions.jsx ← Tombol aksi cepat
│   │   ├── ImagePanel/
│   │   │   ├── BeforeAfterPanel.jsx ← Preview split before-after
│   │   │   └── ImageCanvas.jsx      ← Canvas render gambar
│   │   ├── Controls/
│   │   │   ├── SliderControl.jsx    ← Reusable slider (brightness dll)
│   │   │   ├── FilterPanel.jsx      ← Panel semua filter
│   │   │   ├── SegmentPanel.jsx     ← Panel segmentasi
│   │   │   └── HistogramChart.jsx   ← Recharts histogram
│   │   └── hooks/
│   │       ├── useImageProcessor.js ← Wrapper semua API call
│   │       ├── useHistogram.js
│   │       ├── useUndoRedo.js       ← Stack undo/redo
│   │       └── useFileManager.js     ← Upload & download
│   ├── services/
│   │   ├── api.js                  ← Axios base config
│   │   ├── imageService.js         ← Endpoint image management
│   │   ├── compressionService.js
│   │   └── mlService.js            ← CNN endpoint
│   ├── store/
│   │   ├── imageStore.js           ← Zustand: state gambar aktif
│   │   ├── historyStore.js         ← Zustand: undo/redo stack
│   │   └── uiStore.js              ← Zustand: panel & sidebar
│   ├── App.jsx
│   └── main.jsx
├── .env                            ← VITE_API_URL=http://localhost:5000
├── tailwind.config.js
├── vite.config.js
└── package.json
  
```

4.2 Backend — Python Flask

```

pixelforge-backend/
  app.py                ← Flask app factory & entry point
  config.py             ← Konfigurasi environment
  requirements.txt
  .env                  ← PORT, UPLOAD_FOLDER, SECRET_KEY
  routes/
    __init__.py
    image_routes.py     ← /api/image/* (upload, download, reset)
    enhancement_routes.py ← /api/enhancement/*
    transform_routes.py ← /api/transform/*
    filter_routes.py    ← /api/filter/*
    edge_routes.py      ← /api/edge/*
    color_routes.py     ← /api/color/*
    segment_routes.py   ← /api/segment/*
    compress_routes.py  ← /api/compress/*
    histogram_routes.py ← /api/histogram/*
    ml_routes.py        ← /api/ml/predict
  services/
    image_manager.py    ← Load, save, reset
    enhancement.py      ← Brightness, contrast, histogram eq, sharpen
    geometric.py        ← Affine matrix, rotate, flip, crop, resize
    restoration.py      ← Gaussian, median, salt&pepper; removal
    edge_binary.py      ← Canny, Sobel, Prewitt, Robert, Laplacian, LoG
    color_processing.py ← RGB→HSV, channel split, hue/saturation
    segmentation.py     ← Threshold, edge-based, region-based
    compression.py      ← JPEG quality, Huffman, RLE, LZW
    histogram.py        ← Matplotlib histogram generation
  ml/
    model/
      cnn_model.py      ← Definisi arsitektur CNN
      weights.h5        ← Trained weights
      predictor.py      ← Inference & preprocessing
      labels.json       ← Class label mapping
    utils/
      image_utils.py    ← Base64 ↔ numpy array konversi
      validators.py     ← File type, size validation
      response.py       ← Standard API response format
  uploads/              ← Temporary uploaded files
  temp/                 ← Processed output files
  tests/
    test_enhancement.py
    test_geometric.py
    test_edge.py

```

5. Panduan Setup & Instalasi

5.1 Setup Backend Python

```
# 1. Buat virtual environment
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# 2. Install semua dependensi
pip install flask flask-cors python-dotenv
pip install opencv-python Pillow numpy scipy matplotlib
pip install tensorflow keras

# 3. Buat file requirements.txt
pip freeze > requirements.txt

# 4. Jalankan server
python app.py # Server berjalan di http://localhost:5000
```

5.2 Setup Frontend React (Vite)

```
# 1. Buat project Vite + React
npm create vite@latest pixelforge-frontend -- --template react
cd pixelforge-frontend

# 2. Install dependencies
npm install react-router-dom axios zustand
npm install tailwindcss @tailwindcss/vite
npm install lucide-react recharts
npm install @radix-ui/react-slider react-easy-crop
npm install framer-motion

# 3. Buat file .env
echo 'VITE_API_URL=http://localhost:5000' > .env

# 4. Jalankan dev server
npm run dev # Berjalan di http://localhost:5173
```

5.3 File .env Backend

```
# pixelforge-backend/.env
PORT=5000
DEBUG=True
UPLOAD_FOLDER=uploads
TEMP_FOLDER=temp
MAX_FILE_SIZE=10485760 # 10MB
SECRET_KEY=pixelforge-secret-2024
```


6. Referensi API Endpoint

Semua endpoint menerima dan mengembalikan JSON. Gambar selalu dikirim sebagai Base64 string dalam field **image**. Response sukses menyertakan **result_image** (Base64) dan **status: 'ok'**.

Image Management

Method	Endpoint	Fungsi	Payload
POST	/api/image/upload	Upload gambar awal	{ image: base64 }
POST	/api/image/save	Simpan gambar hasil	{ image: base64, format: 'png', filename: 'output' }
GET	/api/image/reset	Reset ke gambar awal	–

Enhancement

Method	Endpoint	Fungsi	Payload
POST	/api/enhancement/brightness	Atur kecerahan	{ image, value: -100..100 }
POST	/api/enhancement/contrast	Atur kontras	{ image, value: -100..100 }
POST	/api/enhancement/histogram-eq	Histogram equalization	{ image }
POST	/api/enhancement/sharpen	Sharpening filter	{ image, level: 1..5 }
POST	/api/enhancement/smooth	Smoothing / blur	{ image, level: 1..10 }

Geometric Transformation

Method	Endpoint	Fungsi	Payload
POST	/api/transform/rotate	Rotasi gambar	{ image, angle: 0..360 }
POST	/api/transform/flip	Flip gambar	{ image, direction: 'h' 'v' }
POST	/api/transform/crop	Crop region	{ image, x, y, w, h }
POST	/api/transform/resize	Resize gambar	{ image, width, height, interpolation }
POST	/api/transform/translate	Translasi posisi	{ image, tx, ty }

Filter & Restoration

Method	Endpoint	Fungsi	Payload
POST	/api/filter/gaussian	Gaussian blur	{ image, kernel_size: 3 5 7 }
POST	/api/filter/median	Median filter	{ image, kernel_size: 3 5 7 }
POST	/api/filter/noise-removal	Salt & pepper removal	{ image }

Edge & Binary

Method	Endpoint	Fungsi	Payload
POST	/api/edge/detect	Edge detection	{ image, method: 'canny' 'sobel' 'prewitt' 'robert' 'laplacian' 'log' }
POST	/api/edge/threshold	Thresholding binary	{ image, threshold: 0..255 }
POST	/api/edge/morphology	Morphology ops	{ image, op: 'erode' 'dilate', kernel_size }

Color Processing

Method	Endpoint	Fungsi	Payload
POST	/api/color/grayscale	RGB ke Grayscale	{ image }
POST	/api/color/channel-split	Split channel RGB	{ image, channel: 'r' 'g' 'b' }
POST	/api/color/adjust-hue	Atur hue/saturation	{ image, hue, saturation }

Segmentation

Method	Endpoint	Fungsi	Payload
POST	/api/segment/threshold	Threshold segmentation	{ image, threshold }
POST	/api/segment/edge-based	Edge segmentation	{ image }
POST	/api/segment/region	Region segmentation	{ image }

Compression & Histogram

Method	Endpoint	Fungsi	Payload
POST	/api/compress/jpeg	Simulasi JPEG compression	{ image, quality: 1..100 }
POST	/api/histogram/generate	Generate histogram data	{ image }
POST	/api/histogram/compare	Histogram before-after	{ image_before, image_after }

ML / CNN

Method	Endpoint	Fungsi	Payload
POST	/api/ml/predict	Prediksi objek CNN	{ image }
GET	/api/ml/labels	List label kelas	—

7. Contoh Implementasi Kode

7.1 Backend — app.py (Flask Entry Point)

```
from flask import Flask
from flask_cors import CORS
from config import Config
from routes.image_routes import image_bp
from routes.enhancement_routes import enhancement_bp
from routes.transform_routes import transform_bp
# ... import route lain

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
    CORS(app) # Allow React frontend
    app.register_blueprint(image_bp, url_prefix='/api/image')
    app.register_blueprint(enhancement_bp, url_prefix='/api/enhancement')
    app.register_blueprint(transform_bp, url_prefix='/api/transform')
    return app

if __name__ == '__main__':
    app = create_app()
    app.run(debug=True, port=5000)
```

7.2 Backend — services/enhancement.py (Contoh Service)

```
import cv2
import numpy as np
from utils.image_utils import decode_base64, encode_base64

def adjust_brightness(image_b64: str, value: int) -> str:
    img = decode_base64(image_b64) # Base64 → numpy array
    img = img.astype(np.float32)
    img = np.clip(img + value, 0, 255).astype(np.uint8)
    return encode_base64(img) # numpy array → Base64

def histogram_equalization(image_b64: str) -> str:
    img = decode_base64(image_b64)
    if len(img.shape) == 3: # Color image
        img_yuv = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)
        img_yuv[:, :, 0] = cv2.equalizeHist(img_yuv[:, :, 0])
        img = cv2.cvtColor(img_yuv, cv2.COLOR_YUV2BGR)
    else:
        img = cv2.equalizeHist(img)
    return encode_base64(img)

def apply_canny(image_b64: str, low: int = 50, high: int = 150) -> str:
    img = decode_base64(image_b64)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, low, high)
    return encode_base64(edges)
```

7.3 Frontend — services/imageService.js

```
import api from './api'

// Konversi file gambar ke Base64
export const fileToBase64 = (file) => new Promise((resolve) => {
  const reader = new FileReader()
  reader.onload = () => resolve(reader.result.split(',')[1])
  reader.readAsDataURL(file)
})

// Enhancement
export const applyBrightness = async (imageBase64, value) => {
  const res = await api.post('/api/enhancement/brightness', { image: imageBase64, value })
  return res.data.result_image
}

// Edge Detection
export const applyEdgeDetection = async (imageBase64, method) => {
  const res = await api.post('/api/edge/detect', { image: imageBase64, method })
  return res.data.result_image
}

// CNN Prediction
export const predictObject = async (imageBase64) => {
  const res = await api.post('/api/ml/predict', { image: imageBase64 })
  return res.data // { label, confidence, all_scores }
}
```

7.4 Frontend — store/imageStore.js (Zustand)

```
import { create } from 'zustand'

const useImageStore = create((set, get) => ({
  originalImage: null,    // Base64 gambar asli
  currentImage: null,     // Base64 gambar yang sedang diedit
  isProcessing: false,    // Loading state
  history: [],            // Array Base64 untuk undo
  historyIndex: -1,
  setImage: (base64) => set({
    originalImage: base64,
    currentImage: base64,
    history: [base64],
    historyIndex: 0,
  }),
  updateImage: (base64) => {
    const { history, historyIndex } = get()
    const newHistory = history.slice(0, historyIndex + 1)
    set({ currentImage: base64, history: [...newHistory, base64],
      historyIndex: newHistory.length })
  },
  undo: () => {
    const { historyIndex, history } = get()
    if (historyIndex > 0) {
      set({ historyIndex: historyIndex - 1,
        currentImage: history[historyIndex - 1] })
    }
  },
})
export default useImageStore
```

8. Urutan Pengerjaan yang Disarankan

→ Mulai dari backend terlebih dahulu untuk memastikan API berfungsi, baru kerjakan frontend. Gunakan Postman/Insomnia untuk testing endpoint backend sebelum integrasi.

Fase 1 — Fondasi (Estimasi: 2–3 hari)

- Backend: Buat app.py, config.py, dan utils/image_utils.py (Base64 encoder/decoder)
- Backend: Buat route image_routes.py dengan endpoint upload dan save
- Backend: Test endpoint dengan Postman
- Frontend: Setup project Vite + install semua dependencies
- Frontend: Buat api.js (Axios config) dan imageStore.js (Zustand)
- Frontend: Buat komponen upload gambar dan tampilkan di EditorPage

Fase 2 — Fitur Core (Estimasi: 4–5 hari)

- Backend: Implementasi enhancement.py (brightness, contrast, histogram eq, sharpen)
- Backend: Implementasi geometric.py (rotate, flip, crop, resize)
- Frontend: Buat SliderControl.jsx dan FilterPanel.jsx
- Frontend: Buat BeforeAfterPanel.jsx dengan preview split
- Integrasi: Connect frontend ke endpoint enhancement dan geometric
- Backend: Implementasi restoration.py dan edge_binary.py
- Frontend: Tambahkan edge detection controls

Fase 3 — Fitur Lanjutan (Estimasi: 3–4 hari)

- Backend: Implementasi color_processing.py dan segmentation.py
- Backend: Implementasi compression.py dan histogram.py
- Frontend: Buat HistogramChart.jsx dengan Recharts
- Frontend: Tambahkan undo/redo dengan useUndoRedo.js
- Frontend: Buat ToolbarMenu.jsx dengan dropdown lengkap

Fase 4 — CNN & Polish (Estimasi: 3–4 hari)

- Backend: Latih model CNN sederhana dengan TensorFlow/Keras
- Backend: Implementasi ml/predictor.py dan ml_routes.py
- Frontend: Buat panel CNN recognition dengan confidence display
- Frontend: Polish UI — futuristik, dark mode, animasi Framer Motion
- Testing end-to-end semua fitur
- Buat laporan / dokumentasi final

9. Design System & UI Guidelines

PixelForge menggunakan tampilan futuristik dengan dark theme. Palette warna didesain untuk memberikan kesan teknologi tinggi dan modern.

Color Palette

Variabel CSS	Hex	Penggunaan
--bg-primary	#0D1117	Background utama aplikasi (dark navy)
--bg-surface	#111827	Card, panel, sidebar
--accent-cyan	#06B6D4	Aksen utama, highlight aktif
--accent-purple	#7C3AED	Secondary accent, tombol penting
--accent-pink	#EC4899	Highlight spesial, badge bonus
--text-primary	#E2E8F0	Teks utama
--text-muted	#94A3B8	Label, placeholder
--border	#1E293B	Border card dan panel

tailwind.config.js — Konfigurasi Warna Kustom

```
export default {
  content: ['./src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        cyber: {
          bg: '#0D1117',
          surface: '#111827',
          cyan: '#06B6D4',
          purple: '#7C3AED',
          pink: '#EC4899',
          green: '#10B981',
          text: '#E2E8F0',
          muted: '#94A3B8',
          border: '#1E293B',
        }
      },
    },
    fontFamily: {
      mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
    }
  }
}
```

10. Checklist Pengumpulan

✓ Pastikan semua item di bawah terpenuhi sebelum mengumpulkan proyek.

Backend (Python)

- ☐ app.py berjalan tanpa error di localhost:5000
- ☐ Semua 9 modul service terimplementasi
- ☐ Endpoint upload dan download gambar berfungsi
- ☐ Enhancement: brightness, contrast, histogram eq, sharpen, smooth
- ☐ Geometric: rotate, flip, crop, resize, translate
- ☐ Restoration: gaussian, median, salt&pepper;
- ☐ Edge: canny, sobel, prewitt, robert, laplacian, LoG
- ☐ Color: grayscale, channel split, hue/saturation
- ☐ Segmentation: threshold, edge-based, region-based
- ☐ Compression: JPEG quality + minimal 1 metode (Huffman/RLE)
- ☐ Histogram: generate + before-after comparison
- ☐ BONUS: CNN predict endpoint berfungsi

Frontend (React)

- ☐ EditorPage.jsx berfungsi dengan toolbar lengkap
- ☐ BeforeAfterPanel.jsx menampilkan preview sebelum-sesudah
- ☐ Semua filter bisa diaplikasikan via UI
- ☐ Slider bekerja dengan preview real-time
- ☐ Upload dan download gambar berfungsi
- ☐ Histogram ditampilkan sebagai chart
- ☐ Undo/redo berfungsi
- ☐ BONUS: Panel CNN recognition dengan display confidence

Kualitas & Dokumentasi

- ☐ Kode terstruktur sesuai folder struktur di dokumen ini
- ☐ requirements.txt tersedia untuk backend
- ☐ README.md menjelaskan cara menjalankan proyek
- ☐ Tidak ada error/exception saat demo
- ☐ UI terlihat rapi, modern, dan mudah digunakan